



Software Relevant Tools, Standards, and Code Versioning using Github

Name:	Paolo Angelo D. Tolentino	Date:	8/22/2026
Program:	CPE	Section:	B4

A. Learning Outcomes:
At the end of this laboratory, students should be able to:

- 1. Identify software development tools and standards used in software projects.
- 2. Create and manage a GitHub repository.
- 3. Apply Git version control commands.
- 4. Collaborate using GitHub features.
- 5. Demonstrate proper documentation and coding standards.

B. Laboratory Scenario:
You have been hired as a junior software developer in a software company. Your team uses GitHub for source code management and follows coding standards to maintain software quality. Your task is to create a project repository, manage versions of source code, and document your work according to industry standards.

C. Practical Tasks:
1. Software Development Tools and Standards
Create a document named Tools_and_Standards.md containing:
a. List at least 5 development tools.
b. Describe the purpose of each tool.
c. Identify at least three coding standards or best practices used in software development.
(Please include the following: 1. Screenshots of the output)

Screenshot of Output

Software Development Tools and Standards

Development Tools & Purpose

- **Git:** Distributed version control system used to track changes in source code.
- **Visual Studio Code:** Extensible source-code editor for writing, debugging, and building applications.
- **GitHub:** Cloud-based hosting service for Git repositories providing collaboration features like Pull Requests and Issue Tracking.
- **GCC / Clang:** Compilers used to transform C/C++ source code into executable binary files.
- **GDB:** Debugging tool used to inspect and step through software executions to resolve bugs.

Coding Standards & Best Practices

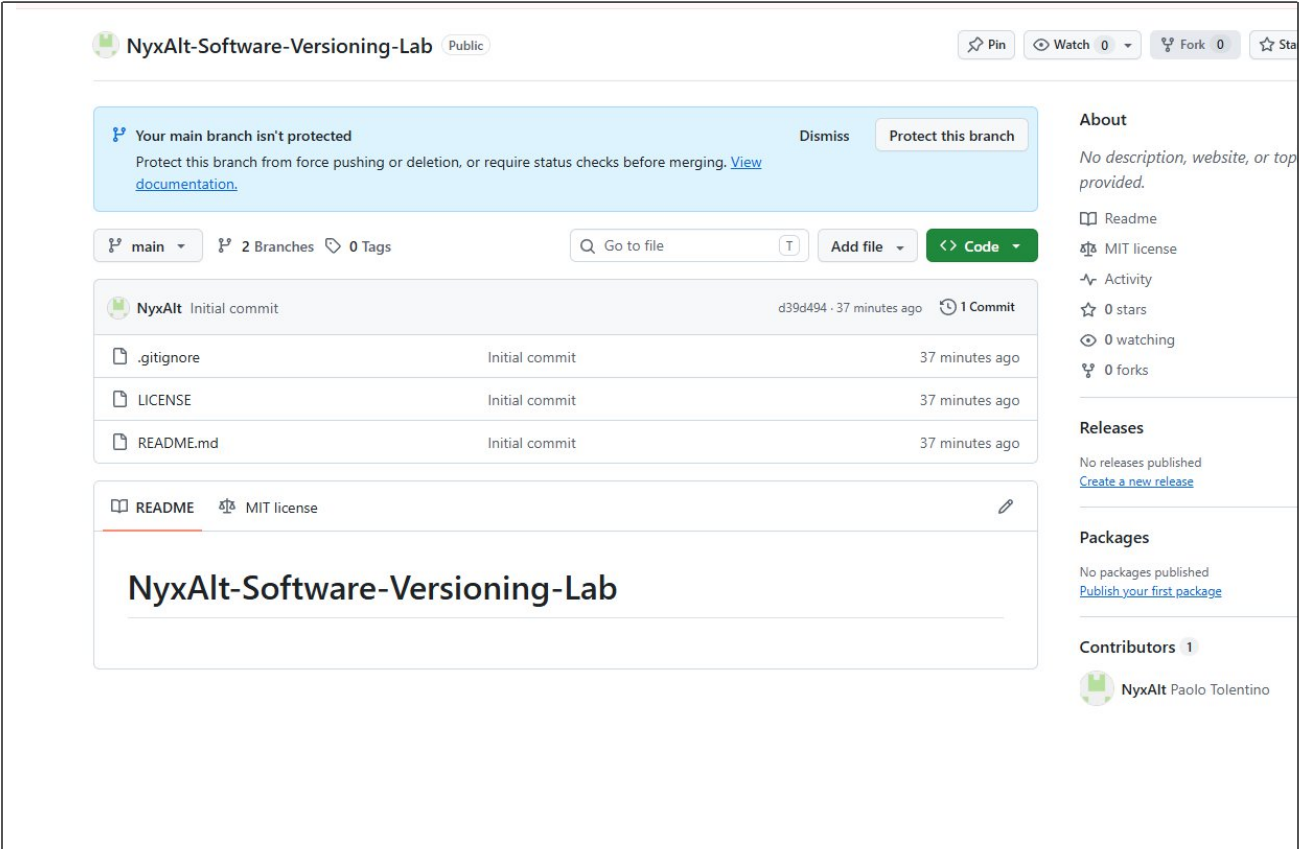
- 1. **Consistent Naming Conventions:** Use clear casing strategies across variables and functions.
- 2. **Proper Indentation and Code Formatting:** Maintain consistent tab spacing to ensure code readability.
- 3. **Comprehensive In-Line Documentation:** Include concise comments explaining non-obvious logic and function headers.

- 2. GitHub Repository Creation
 - a. Create a GitHub account (if none exists).
 - b. Create a public repository named: Software-Versioning-Lab
 - c. Add the following:
 - i. README.md
 - ii. LICENSE
 - iii. .gitignore
 - d. Upload this laboratory file in this repository once accomplished.
 - e. Submit GitHub repository link
 - f. Screenshot of repository homepage

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

Software Relevant Tools, Standards, and Code Versioning using Github



3. Git Version Control
- a. Perform the following Git commands and document them in a file named: (Git_Report.md)

i. Clone Repository

a) git clone <repository-url>

ii. Creating New Branch

a) git checkout -b feature-update

iii. Modify README.md

a) Add your name and laboratory information

iv. Commit Changes

a) git add .

b) git commit -m "Updated README with laboratory information"

v. Push Changes

a) git push origin feature-update

vi. Create Pull Request

a) feature-update → main

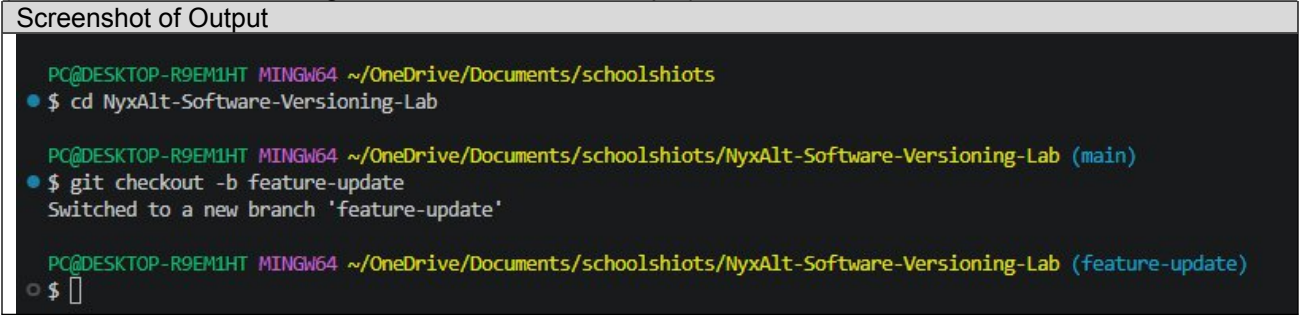
b. Screenshot the following:

i. Commands executed

ii. Pull Request

iii. Commit history

(Please include the following: 1. Screenshots of the output)





Software Relevant Tools, Standards, and Code Versioning using Github

```
PC@DESKTOP-R9EM1HT MINGW64 ~/OneDrive/Documents/schoolshots/NyxAlt-Software-Versioning-Lab (feature-update)
$ git push origin feature-update
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:   https://github.com/NyxAlt/NyxAlt-Software-Versioning-Lab/pull/new/feature-update
remote:
To https://github.com/NyxAlt/NyxAlt-Software-Versioning-Lab.git
* [new branch]      feature-update -> feature-update

PC@DESKTOP-R9EM1HT MINGW64 ~/OneDrive/Documents/schoolshots/NyxAlt-Software-Versioning-Lab (feature-update)
$

```

```
PC@DESKTOP-R9EM1HT MINGW64 ~/OneDrive/Documents/schoolshots/NyxAlt-Software-Versioning-Lab (feature-update)
$ git add .
● git commit -m "Updated README with laboratory information"
[feature-update 061a7e8] Updated README with laboratory information
 1 file changed, 5 insertions(+), 1 deletion(-)

PC@DESKTOP-R9EM1HT MINGW64 ~/OneDrive/Documents/schoolshots/NyxAlt-Software-Versioning-Lab (feature-update)
$

```

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main ← compare: feature-update ✓ Able to merge. These branches can be automatically merged.

Add a title *

Updated README with laboratory information

Add a description

WritePreview

H B I ≡ <> 🔗

≡ ≡ ≡ ≡

📎 🌐 🗑️ ↶ 🗑️

Add your description here...

Markdown is supported

📎 Paste, drop, or click to add files

Create pull request

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

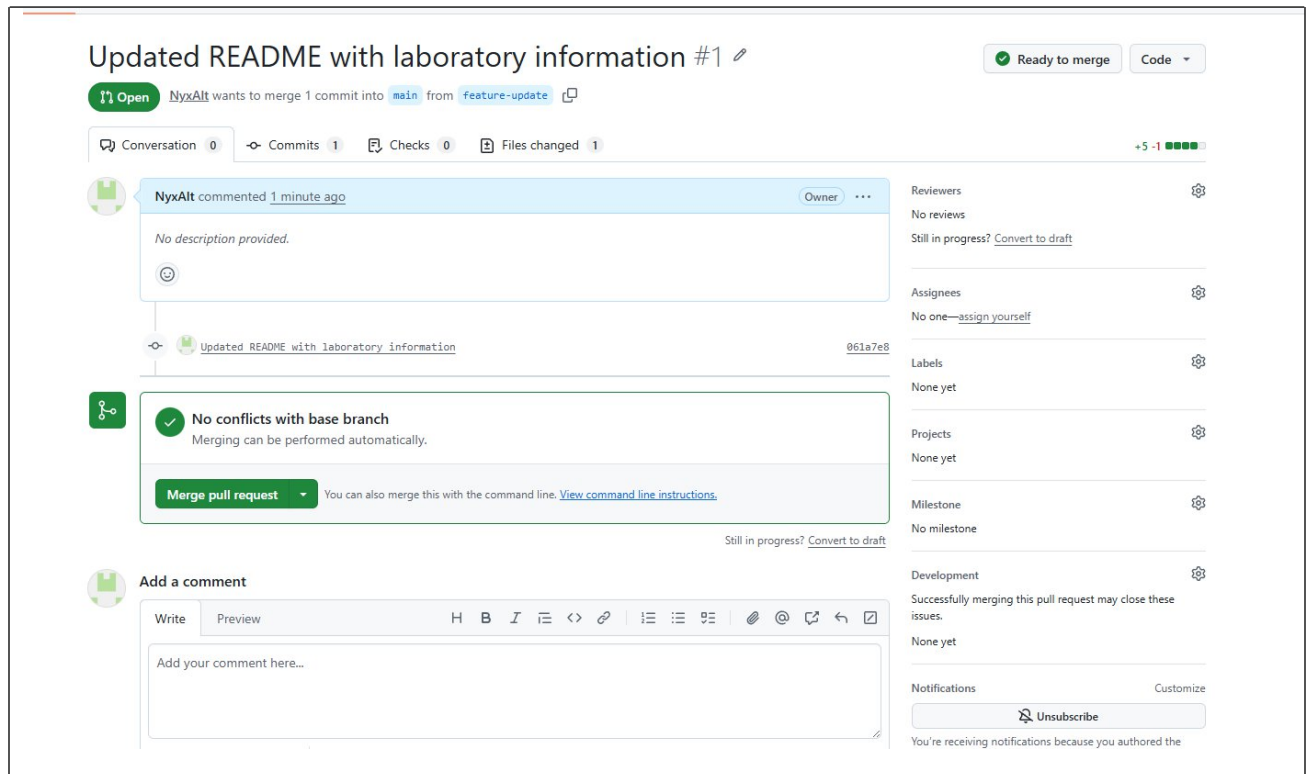
Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Software Relevant Tools, Standards, and Code Versioning using Github



Software Relevant Tools, Standards, and Code Versioning using Github

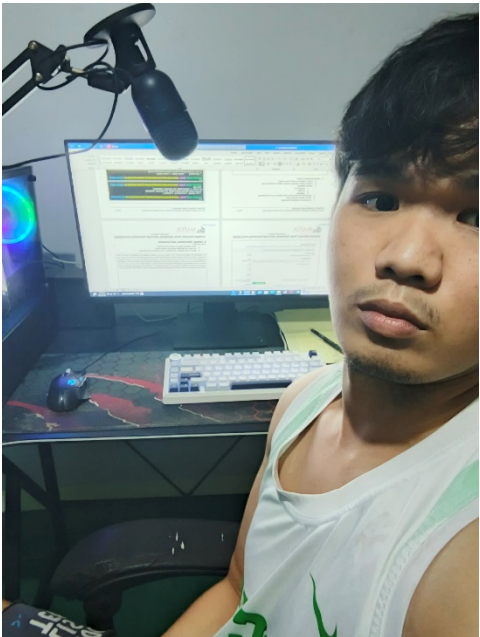
D. Findings, Observations, and Comments:

Guide Questions:

- 1. What additional steps are taken in this activity?
- 2. Are there any difficulties during the GitHub setup?
- 3. What are your findings, observations, and comments in this exercise?

In completing this laboratory exercise, the primary process involved creating and configuring a public remote repository, writing structured documentation using Markdown files, executing standard Git terminal commands, and managing version control operations across isolated branches. No significant difficulties were encountered during the GitHub setup process, as account configuration and remote repository initialization proceeded smoothly. Through this exercise, it was observed that using feature branches prevents direct, unstable edits to the primary `main` branch, ensuring a structured workflow. Furthermore, creating detailed pull requests and maintaining organized commit histories are essential practices in software development that facilitate effective team collaboration and code reviews.

PICTURE/PROOF OF DOING THE ACTIVITY





Laboratory Report 1

Software Relevant Tools, Standards, and Code Versioning using Github

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED

Rating		Signature	
		Date	